Lahore Ride-Hailing

Technical Assessment Response and Implementation Review

Assessment: Lead Full Stack / Backend Engineer take-home Review date: 2 November 2026

Executive summary

The repository implements the core ride-booking assessment as a runnable Lahore demo: a responsive Next.js and TypeScript client, a Go API, Supabase Auth, and durable PostgreSQL state. Riders can request Economy or Comfort trips, review server-calculated estimates, follow ride status, and cancel when allowed. Drivers can control availability, accept eligible rides, and complete assigned trips.

The central reliability requirement is handled in PostgreSQL transactions and constraints, with coverage for competing assignments, retries, lifecycle races, and immutable fare snapshots. Google Maps supports rider-selected pickup and destination pins throughout the configured Lahore service bounds. Distance and fare are transparent estimates; this is an assessment demo, not a production ride-hailing service.

How this review treats the attachment

The supplied PDF is the assessment rubric for this review. Its submission and engineering requirements are treated as criteria to evaluate and answer. The project's direct operating instruction remains to run the Next.js and Go services separately and use the supplied Supabase database; the PDF's alternative mention of Docker does not require a Docker workflow.

Requirement-by-requirement review

Assessment area	Result	Evidence and boundary
Lahore booking flow	Implemented	Seven suggested Lahore stops plus rider-selected pins within Lahore service bounds; Economy and Comfort; server-side PKR fare, approximate distance, pickup ETA, and a fuel/base/distance breakdown; ride confirmation, active status, terminal status, and permitted cancellation. Estimates are illustrative.
Responsive UI and states	Implemented	Rider and driver workspaces include loading, validation, success, empty, conflict, and error handling. Component tests cover quote loading/failure, duplicate-submit suppression, stable retry, and ride states.
Backend API	Implemented	Go/Chi exposes the requested create, read, accept, cancel, and complete operations under `/api/v1`, plus quote, current-ride, locations, driver requests, assignment, availability, health, and readiness routes. See OpenAPI and API contract.
Matching and lifecycle	Implemented	Eligibility checks availability, city, category, and approximate pickup radius. Valid transitions are enforced; invalid transitions return conflicts. Driver occupancy is released transactionally after cancellation or completion.
Durable data model	Implemented	Versioned Supabase migrations create users, driver profiles/categories, locations, rides, assignments, and idempotency records in the private `ride_hailing` schema. Ride state is not held only in memory.
Concurrency and retries	Implemented	PostgreSQL row locks, a consistent ride-then-driver lock order, unique assignment/open-ride/idempotency constraints, and request fingerprints protect state across API processes. The live database race suite is the decisive verification for these guarantees.
Security and configuration	Implemented for the demo	Supabase Auth owns credentials; Go verifies asymmetric JWT signatures through public JWKS and loads role/identity from PostgreSQL. The app checks ownership, role, Origin, and request shape. Local `.env` files are ignored; only example templates are tracked.
Local setup and operations	Implemented for development	The README documents separate Go and Next.js processes, migrations, seed, environment variables, health/readiness, and checks. Docker is not required. There is no hosted production deployment, production monitoring, load test, or backup-restore proof.
Maps	Partial / optional	The configured Google Maps JavaScript API displays suggested stops and accepts rider-selected pickup/destination pins within the configured Lahore bounds. The Go API validates and persists those pins. Distance is Haversine × a configurable 1.3 road factor, not Google road routing. There is no Places search or live driver GPS.
Documentation and submission	Implemented	README, SQL migrations, automated tests, OpenAPI, architecture/state diagrams, design decisions, limitations, and this response cover the requested submission material. AI assistance is disclosed below.

Useful implementation evidence includes booking form, ride page, Go HTTP routes, PostgreSQL ride repository, core schema migration, and Auth identity migration. The database race cases are in ride lifecycle integration tests and idempotency integration tests.

Architecture questions

1. Why this stack and architecture?

I chose Next.js and TypeScript for the rider and driver interfaces, including the same-origin API rewrite and Supabase's cookie-backed browser session. Go with Chi keeps business rules in one explicit API process, while `pgx` gives direct control over PostgreSQL transactions and row locks. Supabase Auth manages credentials and refresh sessions; Supabase PostgreSQL stores application profiles and ride state.

This modular monolith keeps pricing, authorization, matching, and lifecycle behavior in one place, and keeps the UI, API, schema, and contract reviewable in one repository. It also avoids adding distributed infrastructure before the core transaction rules need it. The trade-off is operating separate Node and Go processes and requiring PostgreSQL for the authoritative flow.

2. How do you prevent two drivers from accepting the same ride?

Acceptance starts a PostgreSQL transaction, locks the requested ride row, then locks the acting driver row in a consistent order. After acquiring those locks, the API rechecks ride state, driver availability, city, category, radius, and active assignments. It inserts the assignment and updates the ride and driver together before commit. A unique assignment per ride and a partial unique index limiting each driver to one active assignment provide database-level backstops. A losing request receives a conflict; an in-process mutex is not relied upon.

3. How do you handle retries and duplicate requests?

The browser creates an idempotency key for a booking intent, scopes its saved attempt to the signed-in user, and reuses the key after an uncertain response. The API fingerprints the normalized pickup, destination, and category and persists the key, fingerprint, and ride in the same transaction. Repeating the same request returns the original ride; reusing the key for different input returns a conflict. A unique open-ride constraint also prevents a second active booking under a different key.

For rider-selected coordinates, the normalized endpoint labels and coordinates are included in the fingerprint. The selected points are persisted as non-suggested locations in the same transaction as the ride.

Fare calculation

The server applies a configurable road factor to Haversine distance, then calculates `base fare + fuel cost + distance operating cost`. Fuel cost uses the configured petrol price and vehicle efficiency: `ceil(distance metres × fuel price paisa/litre ÷ efficiency metres/litre)`. Each quote reports the effective date and breakdown; a ride stores the same values as an immutable pricing snapshot. The seeded petrol price is PKR 377.91/litre, effective 31 October 2026 (PSO fuel rates; reported unchanged through 2 November). Petrol prices are manually updated in `ride_hailing.pricing_rates`, rather than fetched automatically. The resulting route distance, fare, and pickup ETA are estimates; they do not use turn-by-turn routing or traffic data.

4. How does the frontend respond when an API request fails?

The interface shows pending states and disables duplicate actions while a request is in flight. Validation and API failures appear as actionable messages. The API client validates each error-envelope field and uses safe fallback text for object-shaped or malformed messages, so a JSON/object payload is not rendered as the login error; a regression test covers this case. On a `401`, the Supabase browser session is cleared; on a `409`, the rider or driver view reconciles with the current server state. If ride creation has an uncertain network result, the client retains the request key and checks the rider's current ride before offering a safe retry. Active rides are refreshed by polling, which stops at a terminal state.

5. How would you scale to multiple cities and thousands of concurrent users?

Keep the Go API stateless behind a load balancer and scale instances horizontally. Keep city and ownership constraints in PostgreSQL, use a managed pooler, inspect query plans, and measure pool saturation, lock waits, and request latency before changing database topology. Add PostGIS when real coordinates and spatial queries replace the current seeded points and approximate distance. Give driver presence a freshness/expiry model before introducing a separate high-write presence store. No load test has yet established capacity for thousands of concurrent users.

6. When would you introduce Redis, a message broker, or separate services?

Use Redis when fresh driver presence and nearby-driver lookup volume make database polling too costly or stale; presence should expire and reconcile back to PostgreSQL. Add a broker when independent consumers need durable ride events, replay, or backpressure. Write an outbox record in the same transaction as each ride transition and make consumers idempotent. Split services only when separate ownership, scaling, or fault isolation outweigh the operational and network-contract cost.

7. How would you deploy, monitor, and roll back a faulty release?

Build immutable web and API artifacts in CI; run unit, contract, migration-compatibility, and PostgreSQL integration checks before release. Apply additive, backward-compatible migrations before deploying code that depends on them, and remove obsolete schema in a later release. Use managed secrets, HTTPS, backups, and tested restore procedures. Preserve the existing request IDs and structured logs, then add latency/error, authorization, pool, lock-wait, and ride-transition metrics with alerts. Roll back application artifacts to a compatible release; use a forward fix for a migration that has already changed durable data.

There is no hosted deployment or production monitoring in the current repository, so this is the proposed operating model rather than a claim about a running service.

8. What are the three most important improvements before production?

1. **Real dispatch inputs and behavior:** replace fixed driver coordinates with authenticated, expiring location updates; add offer expiry and a dispatch loop; use road-aware routing and market-reviewed pricing.
2. **Production identity and abuse controls:** add controlled onboarding and account recovery, a clear revocation/key-rotation policy, rate limits, and a reviewed privacy/data-access model.
3. **Operational readiness:** deploy to managed hosting, add alerts and service metrics, exercise backup restoration and migration rollback procedures, and run representative load tests.

Payment capture and refunds should follow secure dispatch and operational foundations, not be presented as current capabilities.

Verification record

Current local checks

- `go test ./... -count=1` with `TEST_DATABASE_URL` unset: Go unit and HTTP tests passed; PostgreSQL-only cases were skipped in this isolated run.
- `go vet ./...`: passed.
- `npm test`: 23 tests passed across five files.
- `npm run lint`: passed.
- `npx tsc --noEmit`: passed.
- `npm run build`: passed; Next.js generated the production build and routes.
- The full Go suite with `TEST_DATABASE_URL` set to the supplied Supabase PostgreSQL database passed. It exercised the database-backed lifecycle, retry, race, custom-pin, and fuel-price snapshot cases; database test fixtures clean up their rows.
- `npm run test:e2e`: both browser tests passed with real Supabase Auth and Google Maps. A separate ride-journey run against the existing `localhost:3000` app also passed after its API rewrite was refreshed.

Connected-provider evidence

The real Supabase Auth rider-to-driver Playwright journey passed on 2 November 2026, including the driver availability toggle, accepting the specific ride requested, and completion reflected on the rider page. The map test loaded Google Maps, placed custom pins, and displayed the fare breakdown. A database integration test verifies arbitrary in-bounds Lahore pins, the fuel/distance formula, ride snapshots, and exclusion of private pins from suggested locations. Browser journeys use dedicated demo accounts and leave completed/cancelled ride records in Supabase.

AI-use disclosure

AI-assisted coding tools supported implementation and technical documentation. The claims in this response were checked against the repository, and the current local test, lint, type-check, and build results are listed above. Live provider behavior is described only where dated verification evidence exists.

Scope limits

This implementation does not claim free-form address search, road routing, live GPS, dynamic dispatch or reassignment, payment processing, refunds, customer notifications, public signup, OAuth, real-time push updates, automatic fuel-price updates, production deployment, load-test results, or backup/restore proof. Current fare, distance, ETA, and driver positions remain estimates or demo data. See current limitations and architecture notes for the proposed growth path.